iPRA: A Smaller Checker and Two Structural Barriers

A measured synthesis, an explicit Boolean-case separator,
and the exact zero sets of positive-iteration terms

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

The strongest verified positive result of this investigation is a modest implementation reduction for an inherited axiomatization of full first-order intuitionistic primitive recursive arithmetic. A complete checker using ordinary explicit list dispatch has 612 normalized Python bytes and 266 significant tokens. A compact version has 604 bytes; an additional profile using already established Boolean idioms has 593 bytes. The current source corpus attains 616 bytes, 268 tokens, and 300 AST nodes in different programs. After matching rejection drivers, the old envelope is 601 bytes, 264 tokens, and 297 AST nodes. The 593-byte result saves eight bytes against that stricter byte control. It has 315 AST nodes; no delivered program is strictly below every coordinate of that envelope. A guarded parser matches the 300-AST minimum while rejecting strings structurally.

Two additional mathematical results explain why more ambitious deletions fail. First, an explicit formula with two nested progressive binders is provable at both Boolean endpoints but false at a mixed parameter in $\mathbb{N}^2$. This proves that the Boolean case rule in an existing uniform-progressive presentation is not admissible from its other rules. Second, every term built from 0, successor, and positive iteration $h^{n+1}(a)$ has zero set empty, universal, or a single coordinate zero hyperplane. Thus this language cannot term-define predecessor, zero test, ordinary addition, or initialized iteration.

The report supplies every primitive rule, the representation argument, attributed complete arithmetic and logical bridges, reproducible source metrics including gzip level 9, finite validation, and research questions. The corpus is pinned to commit `fb2dbd2a33c6` of ProveIt. The requested significant new axiomatization, an absolute minimum, and a single checker below all current coordinate minima have *not* been established. No proof-assistant verification or literature-wide priority claim is made.

Contents

1 Results and comparison boundary 3

2 The complete inherited axiomatization 5

3 Composing the arity codec with uniform predecessor packets 7

4 An explicit two-binder obstruction to deleting Boolean cases 10

5 Why positive iteration does not immediately remove the computation split 13

6 Validation and reproducibility 15

7 Further research questions 16

Attribution and the inherited complete bridge 17

A Complementary introductions with one local assumption 17

B Arithmetic strength and both ordinary interpretation composites 22

C Internal scalar predecessor bootstrap 24

D From literal-equation induction to full intuitionistic PRA 28

E Pairing, primitive recursion, and internal numerical composites 33

1 Results and comparison boundary

The task asks for a theory that is equiconsistent and bi-interpretable with iPRA, simpler than the supplied drafts by the size of a straightforward proof checker. We use the full quantified intuitionistic theory, not merely its equations or its provably recursive functions. Atomic-induction presentations in the full primitive-recursive language are standard; the inherited bridge here additionally spells out the passage to full quantifier-free induction and the ordinary logical connectives.[7]

1.1 The inspected snapshot

The exact repository snapshot is

`fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`.

Its active `docs/drafts` directory contains 13 ZIP archives. Static recursive inspection encounters 28 distinct archives, 1,383 Python-source occurrences in those distinct archives, and 370 distinct Python contents. Archived Python is parsed for measurement; it is not executed by this census. The supplied inventories retain archive hashes, member origins, source hashes, and all measured source profiles.[1]

The achieved current coordinate minima are 616 normalized bytes, 268 significant tokens, and 300 AST nodes. These are minima among inspected implementations. They are *upper bounds on the unknown optimum*, not mathematical lower bounds on what any possible theory or program requires. Several reports inside the archives quote earlier frontiers; those are not the comparison used here.

The exact leading source programs are retained in `implementation/reference/`. The 616-byte leader is the uniform- predecessor package's ancillary byte checker. Its 300-AST ancillary checker uses generic exception-based iteration and depends on Python's recursion limit to reject nonempty strings. That qualification is kept visible in the comparison.[3]

1.2 Whole-program measurements

Complete implementation	Bytes	Tokens	AST	Gzip 9
Existing byte leader	616	269	311	366
Existing token/AST leader	649	268	300	367
New explicit list dispatch	612	266	310	360
New compact list predicate	604	266	312	356
New Boolean-expression profile	593	267	315	355
New guarded traversal / tuple context	690	269	300	371

The 593-byte source is 23 bytes, about 3.73 percent, below the existing 616-byte source minimum. Its 315 AST nodes exceed the old 300-node minimum by five percent. The preferred explicit-list source is easier to inspect and saves four bytes relative to the best existing byte count. The guarded profile matches the AST minimum and has an immediate type failure for strings; it does not improve the other measures.

All figures count the complete module: parser import and input, syntax opening, scope checks, arithmetic and logical instructions, and verdict handling. Normalization removes genuine docstrings, applies `ast.unparse`, and adds a final newline. Tokens exclude whitespace, comments, and end markers; AST counts traverse the complete module. The common Python interpreter and standard library are the shared platform and are not counted as project source. These are not machine-code sizes or sizes of the entire trusted computing base.

The gzip column is informational only. It uses `gzip.compress(source, compresslevel=9, mtime=0)`, including the gzip wrapper and no embedded filename. Both raw and normalized-source compression measurements and actual compressed checker files are supplied. No gzip-size optimization objective was used.

1.3 Matched controls prevent a misleading improvement claim

The current corpus already permits rejection by nonzero exit status without printing a redundant `REJECT` line. Apply that same driver to the previous arity checker before attributing a gain to the composition:

Matched pair	Bytes	Tokens	AST	Gzip 9
Old arity, explicit lists, same driver	616	265	312	364
New composition, explicit lists	612	266	310	360
Old arity, compact lists, same driver	608	265	314	360
New composition, compact lists	604	266	312	356
Old affine byte leader, same driver	601	274	318	358
Old arity token profile, same driver	641	264	303	364
Old research loop, same driver	698	267	297	380
New Boolean-expression profile	593	267	315	355

Thus the actual packet composition saves four bytes and two AST nodes, but costs one significant token. The two-token gain against the unmodified current 268-token frontier is not evidence of a new logical principle. The smallest source's additional savings use inherited Boolean idioms. The full programs and exact transformations are included so that these distinctions can be checked without trusting a headline percentage.

A broader same-driver screen of 105 complete-program profiles finds an even smaller old control: the affine checker becomes 601 bytes when only its rejection wrapper is removed. Against that fair common-driver minimum, the 593-byte profile saves eight bytes, about 1.33 percent. The 604-byte and 612-byte profiles do not beat that matched minimum. The matched token minimum is 264 and the matched AST minimum is 297; neither is strictly improved by the delivered family. These stricter controls further limit the positive claim to a small source-byte saving. The 264-token and 297-AST controls retain the previously documented recursion-limit-dependent rejection of nonempty strings; they are not structurally terminating parsers on all finite JSON trees.

1.4 New, inherited, and unresolved

The scalar iterator, the Choice-DI logic, the arity codec, the uniform predecessor packet, and the full iPRA bridge are inherited. The code here combines their already established interfaces and verifies that combination. The explicit two-binder separator and the exact positive-iteration zero-profile theorem are additional results relative to the inspected drafts. The product-model method itself is inherited. The present work does not establish worldwide priority for these observations.

The substantial positive target remains open: a genuinely simpler native arithmetic or logical basis with a complete checker below the current frontier under a fair common interface. The two obstruction results are useful constraints on that search; they do not turn the measured source reduction into a foundational breakthrough.

2 The complete inherited axiomatization

This section states the theory whose checking implementation is reduced. Its complete equivalence proof is reproduced in the attributed appendices. No logical axioms are silently supplied by an external theorem prover.

2.1 Numerical syntax and nonlogical equations

There is one numerical sort, with finite terms

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only the update h binds the previous-state variable v. All passive parameters remain fixed. The only numerical computation schemas are

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

Every substitution is capture-free. The update receives its previous value, not the iteration counter.

This binder notation abbreviates a staged first-order signature. For each fixed earlier-stage term $h(v, \vec{p})$ introduce a function symbol $i_h(n, a, \vec{p})$ with the displayed equations. Each term uses finitely many stages. There are no function quantifiers and no total universal evaluator ranging over arbitrary primitive-recursive program codes.

2.2 Formula grammar and all primitive logical rules

Use equations, conditional-universal clauses, and paired formulas:

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

These are interpretations, not additional hidden primitive schemas. Contexts are finite reusable lists. Hypothesis, transport, specialization, and paired elimination are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \quad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}\ \text{DE}.$$

The parameter b in DE is fresh for the retained context, conclusion, and passive matrix parameters. The additional clause is discharged.

There are two complementary paired introductions:

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}\ \text{DI}_{\text{case}},$$

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}E(x,y)} \quad \mathrm{DI}_{\mathrm{progress}}.$$

In both, z is fresh for the retained context, the witness t, and passive matrix parameters. The displayed local assumption is discharged. In the second rule E is a *literal numerical equation*. A formula with a binder above an equation is not eligible for this progress rule.

Computation can be supplied as a theorem or, equivalently, by

$$\frac{\Gamma, H \vdash A}{\Gamma \vdash A} \quad (H \text{ an instance of I0 or IS}). \qquad \text{Supply}$$

There are no additional primitive reflexivity, separation, coverage, excluded-middle, or arbitrary-formula induction rules. Weakening, exchange, contraction, renaming, substitution, and assumption cut are admissible by induction on finite derivations, with eigenparameters freshened before substitution or cut. Substitution preserves the literal- equation classification required by progress introduction.

2.3 Why this is full iPRA

Computation and transport recover reflexivity and the ordinary equality rules. DI-progress, DE, and QE derive contextual induction on a literal equation. DI-case, DE, and QE derive numerical zero/successor cases for arbitrary native formulas. The numerical selector

$$C(n,a,b) = I(n,a;v.b), \quad C(0,a,b) = a, \quad C(Sn,a,b) = b$$

then permits gates and a metasyntactic formula selector. Gate equation leaves by

$$G_z(s=t) = (C(z,0,s) = C(z,0,t)),$$

and recurse beneath binders. After padding unequal binder depths, a structural selector $M_z(A,B)$ satisfies

$$\vdash G_0 A, \qquad G_{Sz}A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \qquad M_{Sz}(A,B) \dashv\vdash B.$$

These operations act externally on fixed finite syntax. They are not internal truth predicates.

For fresh unused u, and a fixed toggle p with $p(1) = 1, p(2) = 0$, the ordinary logical interface is

$$\begin{aligned}
A \Rightarrow_* B &= \mathsf{D}_{u,z} M_z(A,B),\\
A \wedge_* B &= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B),\\
\exists_* x\, A(x) &= \mathsf{D}_{x,y} G_y A(x),\\
A \vee_* B &= \exists_* x\, M_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z),\\
\bot_* &= (0 = S0).
\end{aligned}$$

Every intuitionistic introduction and elimination rule for these macros is proved in Appendix A. The native paired formula itself has the return translation

$$\mathsf{D}_{x,y}B(x,y) \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big).$$

A Q clause has an object representative using a vacuous existential coordinate, with the same return property. This covers native formulas, not only formulas in the image of the ordinary translation.

The numerical bridge proceeds in a fixed order: construct predecessor using scalar iteration, literal-equation induction, and cases; prove actual-equality reflection and full quantifier-free induction;

construct pairing; compile counter-aware primitive recursion. These steps and the internal uniqueness of each fixed recursion are supplied in the appendices. Both numerical interpretation composites have graph $x = y$ on the *entire* numerical domain. No restriction to standard numerals or an unspecified definable cut is used.

For precision, the ordinary first-order realization contains the explicit falsity bridge $(0 = S0) \to \bot$. Positive equations alone admit a singleton structure. Native explosion from $0 = S0$, with ordinary falsity translated to that equation, supplies the required correspondence; it does not justify silently omitting the bridge on the ordinary side. The whole-domain bi-interpretation implies the effective consistency reductions in both directions. These are written proofs, not Lean or Rocq certificates.

3 Composing the arity codec with uniform predecessor packets

The object theory just stated is unchanged. The new implementation combines the arity presentation[2] and the uniform-predecessor instruction protocol[3]. This section discharges their interface obligations rather than assuming that two independently sound programs can be combined by textual substitution.

3.1 Sorted decoding and one opening operation

At formula positions the canonical encodings are

$$s = t \mapsto [s, t], \qquad \mathsf{D}_{x,y}B \mapsto [B], \qquad \mathsf{Q}_y B \mapsto [B, B, B].$$

Only the third field of a general ternary formula has mathematical content. Empty formulas and formula arrays of arity at least four mean $\top_* = (0 = 0)$. Scalar formula padding also means $\top_*$.

At numerical positions zero is represented by 0 or an empty array, successor by $[t]$, and iteration by $[n, a, h]$. Numerical arrays of arity two or at least four mean zero. These are constant aliases, not additional function symbols. The numerical root has depth three, where coordinates $0, 1, 2$ mean zero. Under p protected slots the numerical threshold is $3 + 2p$; coordinates $3 + 2j, 4 + 2j$ select protected slot j. Formula depth under the same slots is $2p$.

Arity	Increment	Formula meaning	Numerical meaning
1	4	D: two bound slots	S: two dummy zero slots
2	3	Equation: change to numerical sort	Constant zero
3	2	Q: one bound slot	I: one state or dummy slot

Thus every array can use the same depth increment $5 - \mathrm{len}(a)$. For iteration, the counter and seed receive a dummy zero slot; its update receives the old-state slot. A successor's two extra slots are dummy zeros. The children of constant aliases may still be traversed and rejected, but have no denotational content.

Negative coordinates name allocated rigid parameters. The global floor k starts at zero and decreases before each shared continuation. A scalar is retained exactly when $k \le a < d$. At an exposed numerical slot it is replaced by the supplied image, which is checked at numerical root depth three. At an exposed formula scalar it becomes a truth alias. Canonical certificates use integer coordinates.

The permissive JSON interface may also admit finite numerical aliases. Positive retained intervals select the same protected slots as their integer representatives; $[0, 3)$ denotes zero. Negative nonintegral values name rigid parameters, identified only when Python numerical equality identifies their labels.

When the floor decreases, all finitely many newly admitted negative labels are fresh. They are not silently identified with the one designated integer eigenparameter. Unused fields need no denotation. Used nonorderable numerical fields fail. Nonfinite numeric values are never retained as names, but can disappear as exposed holes when completed with a checked image; they are not independent numerals in the decoded theory.

Lemma 3.1 (Opening at meaningful positions)**.** *A successful opening call, at the formula or numerical depth just specified, implements capture-free substitution in the sorted decoding. Checked numerical images are independent of external protected slots. The interpretation of a constant alias is unchanged by every successful traversal of its children.*

Proof. Use structural induction on the finite tree. The scalar comparison is exactly the protected-slot and allocated-name condition. The three meaningful arities have the increments in the table. A numerical image checked at depth three has no meaningful exposed positive coordinate; its negative parameters stay free under further binders. On a constant alias, outer arity and its constant interpretation do not change. This proves the assertion for all successful calls. It does not assert raw-tree idempotence underneath ignored large-arity annotations; that stronger assertion is false in the inherited representation. □

3.2 The command format

The public input is a JSON array. Its last entry is the requested formula; earlier entries are four-field packets $[m, s, u, t]$ popped from the end. Membership in the retained context closes a goal without a packet.

For a truthy major m, its proof is checked first. If $m = [s, u]$ and the motive t opened at u equals the goal, the new obligation is the motive opened at s: equality transport. If s is a zero alias and $m = [t, t, t]$, the same endpoint check is QE. An endpoint mismatch retains the original goal. Appending m repeated three times exports its conditional-universal clause when m is a unary paired formula; every other nonempty substantive array gives a harmless truth alias.

A null major selects arithmetic. With $p = s$, seed $a = u$, and update $h = t$, the constructed equation is

$$\begin{cases} I(0, a; v.h) = a, & p \text{ raw-falsy}, \\ I(Sp, a; v.h) = h[I(p, a; v.h)/v], & p \text{ raw-truthy}. \end{cases}$$

The actual successor wrapper is constructed by the checker; raw truthiness is never used as evidence that a submitted number already has successor shape. If the update uses the recursive image, that image is opened and validated at numerical root depth three against the old parameter floor. If the state is unused, no value of that image is needed.

Every other falsy major selects paired introduction. The empty list selects DI-case. For substantive matrices, scalar zero selects DI-progress exactly when the opened matrix is a literal equation, and otherwise DI-case. Even higher-arity truth aliases may take the same harmless progress branch. The witness is t, including literal zero. Positional argument binding in `i(*a,t,2)` permits only a unary substantive paired goal; the empty goal is already a truth alias. The remaining continuation opens the successor goal at the fresh name and appends the selected base or progress assumption.

3.3 Why arithmetic supply is sound in the composed codec

There is a small but consequential interface change. For arithmetic, m is `None`; consequently the final common opener uses image zero, not the freshly allocated integer. Newly admitted rigid negative

labels can nevertheless retain arbitrary values. The computation theorem must hold uniformly in those values.

For a raw-falsy numerical predecessor, the decoded counter is zero and the two occurrences of the decoded seed agree after zero completion; I0 supplies the equation. For a truthy predecessor the constructed unary wrapper denotes an actual successor. If the update demands its state, old-floor validation of $[p, a, h]$ and the opening lemma identify the right side with the update applied to the recursive value. The source's extra dummy zeros change no meaningful parameter: the inserted image is already root-checked, and all exposed positive slots in the final source are completed with zero. Thus IS applies.

If the update does not demand its state, it is independent of that recursive image; IS applies for every decoded predecessor and seed, including newly admitted rigid parameters. High-arity subtrees contribute only their fixed zero interpretations. This proves uniform validity in the new parameters, rather than testing just their zero valuation.

The older arity packet could open a generated hypothesis at a fresh parameter and therefore used a known-zero-instance discharge theorem. The new null tag has the stronger uniform-computation property just proved. The two arguments should not be conflated. The independent diagnostic checks newly allocated parameters at $0, 2, 5$ as a finite implementation check of this distinction.

3.4 Soundness, completeness, and resource qualification

Theorem 3.2 (Composed verifier)**.** *Every successful execution of any of the four supplied complete checker profiles has a decoded conclusion derivable in the native theory of Section 2. Every closed native theorem has a finite accepted canonical certificate when sufficient computation resources are available; open theorem schemas can be represented by their derived universal closures. The resulting object theory is therefore equiconsistent and whole-domain bi-interpretable with full first-order iPRA.*

Proof. Induct on the finite successful checking trace. Hypothesis membership is justified by raw equality and the opening lemma. A proved equality or closure major implements Tr or QE when the endpoints match; a mismatch supplies only a neutral continuation. Unary major triplication implements DE at a fresh parameter. Other major exports denote truth. The exact positional-argument check makes substantive introduction a genuine DI instance, with the literal-equation restriction retained. Arithmetic supply is an I0 or IS theorem by the preceding argument. All parameters admitted only inside a completed major proof and unused by its end sequent can be replaced by zero; they carry no retained assumptions into the continuation. Goal and witness validation before fresh allocation establish the eigenvariable conditions.

For completeness, encode a closed native derivation structurally. Its logical rules have the canonical arity encodings above, and literal zero is now an allowed DI witness. I0 is direct. IS at a raw nonzero predecessor is direct. For IS at literal zero, use the nonempty term $W(0) = I(0, 0; v.0)$, prove $W(0) = 0$ by I0, apply the direct successor packet at $W(0)$, and transport along that equality. This is a finite expansion, not a restriction to nonzero numerical predecessors.

The compact and explicit-list profiles differ only in their JSON list predicate. The short profile's false-valued replacement fallback can only arise at a truth alias, so it preserves soundness and canonical completeness. The guarded profile traverses `a+[]` inside a protected block: only a list can take that branch on JSON input. Immutable tuple contexts prevent a major proof from mutating its caller's context. Its tail loop is the same shared continuation. All four therefore implement the stated calculus. The final metatheorems follow from the attributed object-level bridges, not from finite test success. □

On finite JSON trees the opener either traverses a proper subtree or checks an image with the missing-image sentinel as its next default. Every non-hypothesis proof node consumes a packet. Thus ideal checking terminates; strings cannot create the recursive-alias behavior of the old generic-iteration AST profile. Physical Python recursion or memory limits may still reject very large valid certificates. Completeness is not a promise of bounded proof size or bounded resource use.

3.5 Entire preferred source

The following is the complete 612-byte explicit-list checker. The 593-byte profile is supplied separately, not silently substituted for this easier-to-read source.

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k
    <= a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t] * 3) and i(t, u) == a:
            a = i(t, s)
        h = m * 3
    elif m is None:
        h = [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]
    else:
        h = a = i(*a, t, 2)
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

Success means exit status zero and exactly the line `ACCEPT`. An exception rejects with nonzero exit status; its traceback is a diagnostic. Unread input prefixes have no logical role. Input is JSON data and is never executed as code. No checker imports the producer, an arithmetic evaluator, an axiom table, or a theorem library.

4 An explicit two-binder obstruction to deleting Boolean cases

The progressive-binder alternative in the supplied research draft uses

$$H_{x,y}B = \exists x\, \forall y\big(B(x,y) \to B(x,Sy)\big)$$

as its ordinary reading. The following small formula shows that its Boolean case rule cannot simply be deleted. The countermodel is the direct-product construction already available in the draft family; the explicit native separator identifies exactly where that model defeats the progressive interface.

4.1 The precise case-free calculus

Numerical terms are variables, zero, successor, and fixed staged counter-aware primitive-recursive descriptions $R(n, a; r, v.h)$. The two computation schemas are

$$R(0, a; r, v.h) = a, \qquad R(Sn, a; r, v.h) = h[n/r, R(n, a; r, v.h)/v].$$

Only r, v are bound in h, other parameters are fixed, and the successor substitution is simultaneous and capture-free. Object formulas are equations $s = t$ and paired formulas $H_{x,y}B(x, y)$. Internal clauses $P_yB(y)$ have the ordinary reading $\forall y(B(y) \to B(Sy))$; they may occur in contexts but are not object goals. Write $\Gamma \triangleright L$ when L is a context member or is a proved object. This is availability, not a primitive rule concluding an arbitrary assumed object.

In addition to the computation schemas, the complete case-free primitive rule inventory is

$$\frac{\Gamma \triangleright s = t \qquad \Gamma \vdash A(s)}{\Gamma \vdash A(t)}\ \mathrm{Tr}, \qquad \frac{\Gamma, B(t, z) \vdash B(t, Sz)}{\Gamma \vdash H_{x,y}B(x, y)}\ \mathrm{HI},$$

$$\frac{\Gamma \triangleright H_{x,y}B(x, y) \qquad \Gamma, P_yB(a, y) \vdash A}{\Gamma \vdash A}\ \mathrm{HE},$$

$$\frac{\Gamma \triangleright P_yB(y) \qquad \Gamma \vdash B(t)}{\Gamma \vdash B(St)}\ \mathrm{Step}, \qquad \frac{\Gamma \triangleright P_yE(y) \qquad \Gamma \vdash E(0)}{\Gamma \vdash E(t)}\ \mathrm{Iter}.$$

Here E in Iter is a literal equation. The variable z in HI is fresh for the retained context, the witness t, and the passive matrix parameters. The variable a in HE is fresh for the retained context, the conclusion, and the passive parameters. The displayed local assumptions and clauses are discharged. Contexts are reusable; weakening, exchange, contraction, renaming, capture-free substitution, and assumption cut have the usual external derivation inductions. Neither unrestricted formula induction nor any case rule is included.

Derived object identity, without cases. A computation theorem $R(0, s; r, v.0) = s$, used both as a major equation and as the minor proof for transport, gives $s = s$. An available equation $s = t$ then transports $s = s$ to $s = t$. Induct externally on formula depth for an available object $H_{x,y}B$. Open it by HE, obtaining a fresh a and $P_yB(a, y)$. Reintroduce the same object by HI with witness a. Its local assumption $B(a, z)$ yields a proof of itself by the induction hypothesis, and Step gives $B(a, Sz)$. Discharge HI and HE. This derives $\Gamma, A \vdash A$ for every object A without Boolean cases. Subsequent uses of identity below mean this finite expansion, not an extra primitive hypothesis rule. Symmetry, transitivity, and formula transport have their usual derived equality expansions.

The deleted rule, for comparison, is

$$\frac{\Gamma, \mathrm{sg}(t) = 0 \vdash A \qquad \Gamma, \mathrm{sg}(t) = 1 \vdash A}{\Gamma \vdash A}\ \mathrm{BoolCases},$$

where $\mathrm{sg}(0) = 0$ and $\mathrm{sg}(Sn) = 1$. Its conclusion may be any native object formula.

4.2 The native separator

Let C and Z be fixed primitive-recursive descriptions satisfying

$$C(0, a, b) = a, \quad C(Sn, a, b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0.$$

For a free numerical parameter t, put

$$\boxed{F(t) := H_{u,y}\, H_{v,z}\big(C(t, Z(y), z) = 0\big).} \qquad (*)$$

The existential coordinates u, v are unused, and all four displayed bound variables are distinct.

Theorem 4.1. *In the progressive calculus without Boolean cases, $F(0)$ and $F(1)$ are derivable. Nevertheless the schema $F(\mathrm{sg}(t))$ is not derivable. Consequently the Boolean case rule for arbitrary native conclusions is not admissible in that calculus, even in the presence of all primitive-recursive symbols, their defining equations, and contextual literal-equation induction.*

Proof. First consider the endpoint derivations. At $t = 0$, the inner matrix reduces by computation to $Z(y) = 0$, independently of z. Its progress implication is therefore identity, so inner introduction proves $H_{v,z}(Z(y) = 0)$ for arbitrary y. Outer introduction may ignore its local assumption. At $t = 1$, the inner formula reduces to $H_{v,z}(z = 0)$, independently of y. Outer progress is then identity. All witnesses can be zero. These arguments use only derived object identity, computation/transport, and progressive introduction.

For nonderivability use the classical one-world model $\mathcal{M}$ with domain $\mathbb{N}^2$ and every primitive-recursive function interpreted coordinatewise. Zero is $(0, 0)$, successor adds one in each coordinate, and equality is actual equality of pairs. All numerical computation laws and constructor separation hold.

Every literal-equation induction axiom also holds. For a fixed equation $E(n, \vec{p})$ and fixed parameter pairs, let

$$U_j = \{r \in \mathbb{N} : E(r, \vec{p}_j) \text{ holds in } \mathbb{N}\}, \qquad j = 0, 1.$$

The base premise says $0 \in U_0 \cap U_1$. If $r \in U_0$, the pair $(r, 0)$ satisfies E; the step premise implies that $(r + 1, 1)$ satisfies E, so $r + 1 \in U_0$. Similarly $(0, r)$ proves successor closure of U_1. External ordinary induction gives $U_0 = U_1 = \mathbb{N}$, proving the asserted induction instance. This proves the full schema with parameters and hence validates its contextual rule under arbitrary retained assumptions.

All the remaining progressive-calculus rules are sound in this model under the displayed ordinary reading of H: unrestricted progressive introduction and elimination are logical rules, and the exceptional zero-to-arbitrary specialization for equations is justified by the induction schema just checked.

Take $e = (0, 1)$, so $\mathrm{sg}(e) = e$. At this parameter the inner equation in $(*)$, for $y = (y_0, y_1)$ and $z = (z_0, z_1)$, is

$$(Z(y_0), z_1) = (0, 0).$$

If $y_0 = 0$, this equation is false for every z, so every inner progress implication has a false antecedent. The inner H is true. If $y_0 > 0$, the equation says $z_1 = 0$. It is true at $z = (0, 0)$ and false at $Sz = (1, 1)$, so the inner H is false. Thus the inner formula is true at $y = (0, 0)$ and false at $Sy = (1, 1)$, which falsifies the outer progress assertion. Therefore $\mathcal{M} \not\models F(e)$. Soundness gives the stated nonderivability.

Finally, from a retained assumption $\mathrm{sg}(t) = 0$ the endpoint theorem $F(0)$ and transport give $F(\mathrm{sg}(t))$. The other branch follows from $F(1)$ in the same way. An admissible Boolean case rule would therefore derive the refuted schema. This contradiction proves that the rule is not admissible. □

The binder depth is sharp for this product separation. A numerical equation that is true at both standard endpoints remains true at their product parameter $(0, 1)$. The same is true of a formula with just one progressive binder and an equational matrix: combine the two standard existential witnesses into a pair, and verify each progress implication coordinatewise. Thus no formula of progressive binder depth at most one can give this particular endpoint/product separation. Formula $(*)$ has depth two. The first failure arises when an implication tests a compound premise: one coordinate can make an inner implication vacuous while another coordinate would fail it.

Inherited obstruction for ordinary bi-interpretations. The supplied research draft [4] also establishes a stronger model-theoretic obstruction, applicable here with all hypotheses checked. Let $\mathcal{E}$ be the eventually constant sequences in $\mathbb{N}^{\mathbb{N}}$, with all fixed PR operations interpreted pointwise. The same coordinate proof of equation induction applies, using a sequence that equals r at one coordinate and zero elsewhere. Hence $\mathcal{E}$ validates all the case-free rules above under the displayed ordinary H, P readings. These readings are first-order formulas, so their truth is invariant under every automorphism of the numerical structure.

Any finite tuple of parameter sequences is constant outside a finite set of coordinates. Swapping two coordinates outside that set gives a nonidentity automorphism of order two fixing those parameters. For an ordinary finite-dimensional definable interpretation, including a quotient interpretation, automorphisms fixing its finite parameter data act on representing tuples and induce automorphisms of the interpreted structure. A definable source-side return isomorphism makes that action injective: an automorphism acting trivially on the interpreted structure also acts trivially on the returned copy, and the definable isomorphism then forces it to fix every original element. Parameters used by the return isomorphism and finite representatives of reverse-interpretation parameters can all be included in the initial finite tuple.

Every ordinary iPRA model has a definable strict linear order, for example $\ell < m$ iff $S\ell \dot{-} m = 0$. An order-preserving automorphism of finite order is the identity, since either strict inequality between an element and its image would produce a strict cycle. An injective image of the coordinate transposition is therefore impossible. Thus the case-free system, under its ordinary first-order realization, cannot be repaired by an ordinary definable bi-interpretation with finite parameter data merely by changing to a finite-dimensional domain or quotient. This is an application of the supplied draft's existing automorphism theorem, not a new theorem claimed by this report. It is separate from the explicit two-binder diagnostic above.

Scope. This is a direct rejection of the proposed deletion of Boolean cases, not a lower bound on checker size and not an impossibility theorem for other logical bases, other primitive operations, or arbitrary interpretations. The product-model method and the progressive-binder presentation are inherited from the supplied drafts. The explicit formula $(*)$ is an additional diagnostic; no worldwide novelty claim is made.

5 Why positive iteration does not immediately remove the computation split

Consider the staged numerical term language with variables, 0, successor, and the positive iterator

$$R^+(n, a; v.h) = h^{n+1}(a).$$

All passive parameters of h are fixed during iteration. The defining equations are

$$R^+(0, a; v.h) = h[a/v], \qquad R^+(Sn, a; v.h) = h[R^+(n, a; v.h)/v].$$

This is not the tied-seed language considered in the existing drafts: $R^+(n, a; v.Sv) = n + a + 1$ has unbounded dependence on both n and a.

Zero-profile theorem. For each fixed term $t(x_1, \ldots, x_r)$, its zero set in the standard natural-number interpretation is exactly one of the following:

$$\varnothing, \qquad \mathbb{N}^r, \qquad \{\vec{x} : x_j = 0\} \quad \text{for a single } j \in \{1, \ldots, r\}.$$

The profile and its coordinate, when present, are effectively obtained by structural recursion on the finite term.

Proof. Zero is always zero, a successor term is never zero, and a variable x_j has zero set $x_j = 0$. Consider an iterator whose update is $h(v, \vec{x})$. By structural induction its update has one of four kinds of zero profile. If the update is never zero, then the iterator is never zero. If the update is always zero, then the iterator is always zero. If the update is zero exactly when a single passive variable x_j is zero, then the iterator has that same zero profile. Each of these three cases uses the fact that at least one update occurs. Finally, if the update is zero exactly when its state variable v is zero, each update preserves the zero status of the current state. The final state therefore has exactly the seed's zero profile, already classified by structural induction. The count's zero profile is not used: the number of updates is always positive. This completes the external structural induction, including arbitrarily nested update bodies. □

Consequences. Every unary term has zero set equal to $\varnothing$, $\{0\}$, or all of $\mathbb{N}$. Consequently no such term defines predecessor, whose zero set is $\{0, 1\}$, nor the zero-test that is 1 at zero and 0 at every positive number. An unrestricted initialized iterator cannot be compiled by terms: its constant-update special case is the selector $I(0, a; v.b) = a$, $I(Sn, a; v.b) = b$, and substituting $a = 1, b = 0$ would give the forbidden zero-test. Ordinary binary addition is also not term-definable: its zero set is $\{(x, y) : x = 0 \text{ and } y = 0\}$, which is neither a single coordinate hyperplane nor empty nor the whole domain. The definable shifted addition $n + a + 1$ does not contradict this conclusion, because it is never zero.

Exact scope. This is a standard-model term-definability obstruction, not an impossibility theorem for arbitrary relational interpretations or changed domains. In ordinary intuitionistic first-order logic with successor coverage, injectivity, and separation $0 \neq Sm$, the formula

$$(n = 0 \wedge z = a) \ \vee \ \exists m \, (n = Sm \wedge z = R^{+}(m, a; v.h))$$

defines a total unique graph for the initialized iteration of each fixed old-language update h. This observation by itself does not give a bi-interpretation with iPRA. One must still prove induction for the graph-defined expanded language and closure when newly defined updates are nested inside iteration. In the inherited Eq/Q/D native calculus, even the ordinary logical connectives in this graph formula are reconstructed using a selector, so assuming them at that point would additionally be circular.

Counter variables that also occur as passive variables in h, or occur in the seed a, remain present in those respective profiles; only their separate role as the iteration count is irrelevant to the zero-profile step.

Implementation motivation. A proposed raw encoding gives a two-field numerical array $[a, h]$ the value a and a three-field array $[n, a, h]$ the value $R^{+}(n, a; h)$. For a zero-or-successor counter the computation can then be written with a uniform predecessor expansion rather than a zero/successor conditional. The zero-profile theorem explains why this apparent source reduction cannot simply inherit the existing term compiler or selector-based logical bootstrap.

6 Validation and reproducibility

The supplied programs and diagnostics use Python's standard library. The recorded environment and exact file hashes are in `evidence/environment.json` and the package manifest. Reproduction checks the frozen source metrics and gzip files before executing the scoped tests. A different Python or zlib version may change normalized or compressed bytes, so those versions must accompany any later result.

Check	Recorded result
42 certificates and 10 malformed inputs; four profiles, normal and `-O` Python	416/416
Four deliberately unsafe mutations, both Python modes	8/8 controls
Generated arithmetic equation valuations	55,443
Independent binder/substitution comparisons	216
Positive-iteration terms, including nested examples	2,868
Positive-iteration numerical valuations	45,888
Arithmetic or profile evaluation skips	0

The certificate suite includes I0, state-dependent and state-independent IS, the zero-predecessor expansion, literal-zero witnesses, a nonvacuous witness round trip, transport below binders, contextual equation induction, and compound case introduction. Negative cases exercise unallocated names, forbidden compound progress, false equations, false majors, wrong endpoints, malformed node shapes, and bad input types. The unsafe mutations demonstrate that the guards reject specific prohibited certificates; they are not evidence that every possible bug has been excluded.

The arithmetic diagnostic considers 24,050 source triples and evaluates 55,443 generated equations after rejecting invalid supplies. It varies the newly allocated parameter over three values. The zero-profile diagnostic independently evaluates every well-scoped term through seven syntax nodes with two free variables, plus three additional distinct nested examples. It compares semantic numerical evaluation with the structural zero-profile extractor, including shared counter/passive variables. A finite product-model script illustrates the explicit progressive-binder separator.

These are finite implementation checks. Universal soundness, completeness, the obstruction theorems, and bi-interpretability rest on the written arguments. There is no Lean/Rocq formalization of the complete metatheory, and the named producer is not a general automatic compiler from arbitrary ordinary-iPRA derivations. The reviews in `evidence/` are independent assistant reviews, not external human peer review.

From the extracted package run

```
python3 reproduce.py
python3 build.py
```

The first command verifies the distributed payload and reproduces the measurements and finite evidence. The second compiles the report using pdfLaTeX. For a single primitive proof:

```
python3 implementation/checker_explicit.py < \
  implementation/fixtures/I0_arbitrary_seed.json
```

To repeat the full source census against a checkout of the pinned corpus:

```
python3 code/audit_corpus.py --archives /path/to/ProveIt/docs/drafts
```

The archived corpus itself is not recursively duplicated in this package; its exact source identities and all comparison programs used in the reported tables are included.

7 Further research questions

A Boolean principle with a cheaper interface. The explicit progressive separator identifies the missing behavior at binder depth two. Can a primitive rule weaker in syntax than arbitrary Boolean cases supply exactly the necessary mixed-parameter elimination, while admitting a complete checker smaller than Choice-DI? A proposed rule must defeat the displayed $\mathbb{N}^2$ model and must support both native formula round trips. Merely recovering an ordinary-language image does not settle bi-interpretability of the original native theory.

An escape from the positive-iteration zero profile. Adding a zero-test primitive immediately destroys the projection-only zero invariant. For example, $R^+(n, 0; v.Zv)$ alternates, so zero status can depend on the iteration count. Can a small independently justified operation be supplied more cheaply than the original base/step split, and can it recover a selector and the complete native logical bootstrap? The added operation's defining equations and checker branches must be counted.

Relational initialization without stronger hidden induction. The graph of initialized iteration is definable by zero/successor cases from positive iteration when ordinary logic and constructor axioms are already present. Is there a native derivation that avoids assuming the selector-based logic it is meant to reconstruct? How does induction transfer through the graph definitions when newly defined updates are nested? A proof of total unique graphs alone is not enough.

A simultaneous source improvement. The new byte and token counts leave the AST frontier unbeaten. A useful next target is one structurally terminating complete program below 593 bytes, 264 tokens under matched-driver controls, and 297 AST nodes, with the same counted trust boundary. This is an empirical target, not a conjectured absolute lower bound. A genuinely new invariant or rule interface is more promising than another rearrangement of Python expressions.

Machine-checking the interpretation boundary. The most valuable formalization target is the native-to-ordinary bi-interpretation, including formula composites, freshness, arbitrary retained contexts, and the scalar predecessor bootstrap. It would make later representation changes easier to review and would separate a small trusted verifier from an increasingly long handwritten metatheory.

Certificate growth and useful complexity. Triplicated closures can make explicitly serialized nested certificates grow as 3^r in closure depth. The current metric charges checker source, not certificate size or checking time. A shared representation may be worth several additional checker bytes if it yields much smaller proof objects. The comparison should state the resulting tradeoff rather than equating a source record with an overall simpler proof system.

Attribution and the inherited complete bridge

The next five appendices reproduce the mathematical bridge supplied in the arity-calculus archive at the pinned repository commit. They are inherited proofs, not new claims of this report. Their original attribution is retained, and `reference/provenance.json` records exact source hashes. The repository's MIT No Attribution license is included. The ordering below reflects the dependency chain; the ordinary interpretation summary refers forward to the detailed arithmetic proofs.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the

progress choice. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [9]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}(K(d(Sa,b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (39) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,\big(a=b\vee(a=b\to\bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{42}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{43}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{44}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A=0\vee b_A=1\big)\quad\text{and}\quad\big(b_A=0\longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [10]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, active draft collection*, commit `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`. https://github.com/VladimirReshetnikov/ProveIt/tree/fb2dbd2a33c6b132671a54b029d0bf3cd98c633d/docs/drafts.

[2] *An Arity Calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip` in [1]. Source of the unary-D / triplicate-Q codec, the depth law, semantic opening argument, and the five inherited proof appendices reproduced here.

[3] *Uniform Predecessor Certificates for Intuitionistic PRA*, `ipra_uniform_predecessor_2026-11-10.zip` in [1]. Source of uniform predecessor packets, the null arithmetic tag, counted source controls, and the named proof producer adapted here.

[4] *iPRA research report*, `ipra_research_2026-11-10.zip` in [1]; especially `theory/progressive_binder.tex` and `theory/finite_parameter_obstruction.tex`. Source of the complete progressive-binder presentation with Boolean cases and the inherited product/automorphism obstruction method.

[5] *Affine Depths for a Small iPRA Checker*, `ipra_affine_arity_2026-11-10.zip` in [1]. Source of related alias-decoding comparisons and inherited Boolean implementation idioms; its arithmetic and logic remain unchanged.

[6] *A One-State Recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip` in [1]. Its atomic-case obstruction warns that a restricted image translation does not supply a native formula round trip.

[7] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*. Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[8] Inherited scalar predecessor development, retained with original attribution in `reference/scalar_predecessor.tex` of [2].

[9] Inherited forward-left equality-reflection development, retained in `reference/equality_reflection.tex` of [2].

[10] Inherited progress-closure pairing and primitive-recursion compiler, retained in `reference/primitive_recursion.tex` of [2].